Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 1

EXP-0 «Вводное задание: эксперимент»

по дисциплине «Операционные системы»

Вариант: Write, Cache, 10M vs 1M, Rand

Выполнил:
Рахаман Мд Афифур
группа P3330, ИСУ 407793

Преподаватель:
Клименков Сергей Викторович

Санкт-Петербург, 2026

Содержание

1 Цель работы и вариант 3
2 Ход работы 3
2.1 Паспорт системы (шаг 4.1) 3
2.2 Подготовка окружения (шаг 4.2) 4
2.3 Тестовые данные и сборка 4
2.4 Ознакомительные измерения (шаг 4.3) 4
2.5 Гипотеза (шаг 4.4) 5
2.6 План эксперимента (шаг 4.5) 5
3 Результаты 6
3.1 Этап 1: `graph_traverse` (read/lseek/write) 6
3.2 Этап 2: `graph_traverse_mmap` 7
3.3 Этап 3: все серии вместе 8
3.4 Интрузивность измерительных инструментов 9
4 Заключение 10
5 Приложение А. Воспроизведение 11
6 Приложение Б. Удалённые выбросы (1,5 × IQR) 11
7 Приложение В. Скрипт сбора `collect.sh` 11

1 Цель работы и вариант

Цель. Экспериментально сравнить время работы выданных прототипов приложений на модифицирующей нагрузке и научиться корректно ставить эксперимент над ОС: фиксировать и описывать окружение, выбирать инструменты, планировать число измерений и статистически обрабатывать результаты.

Вариант «Write, Cache, 10M vs 1M, Rand»: обе программы работают в режиме записи (`--write`, каждая посещённая вершина изменяется), используется системный страничный кэш (без `--no-cache`), граф — цепочка со случайным физическим порядком (`--topology chain`), сравниваемый фактор — **размер файла**: 1 МиБ и 10 МиБ. Вопрос варианта: **важен ли размер в типичном случае на современной системе?**

Объекты исследования: `graph_traverse` (доступ через `lseek()` + `read()`/`write()`) и `graph_traverse_mmap` (файл отображается в память через `mmap()`).

2 Ход работы

Все скрипты, сырые данные, логи и графики: https://github.com/simu18/os-course/tree/lab-intro-exp/lab/intro-exp (ветка `lab-intro-exp`). Любое число ниже воспроизводится командами из приложения А.

2.1 Паспорт системы (шаг 4.1)

Эксперимент проводился в виртуальной машине. Гостевая ОС видит только виртуальные устройства, поэтому физические параметры взяты с хост-системы и помечены соответственно.

Параметр	Значение	Источник
Хост	MacBook Air, Apple M2: 8 ядер (4 производительных + 4 энергоэффективных), 16 ГБ ОЗУ, macOS 26.6.2	`system_profiler` на хосте
Гипервизор	UTM, бэкенд QEMU	`systemd-detect-virt` = `qemu`
Гостевая ОС	Ubuntu 26.04.1 LTS, ядро 7.0.0-38-generic, aarch64, PREEMPT_DYNAMIC	`uname -a`, `os-release`
vCPU	4 vCPU, 1 поток на ядро, 1 сокет; производитель «Apple», модель скрыта («-»); размеры кэшей не сообщаются	`lscpu`, `lscpu -e`
Модель памяти	1 узел NUMA, т.е. **UMA**	`lscpu`, `numactl --hardware`
ОЗУ	выделено 4 ГиБ, доступно 3,3 ГиБ; swap 3,8 ГиБ	`lshw`, `free -h`
Размер страницы	4096 Б	`getconf PAGESIZE`
Диск	virtio `vda`, 25 ГБ, ext4; планировщик ввода-вывода `none`; физически — файл-образ на SSD хоста	`lsblk`, `df -hT`, хост
Тип диска	гостевая ОС сообщает `rotational = 1`, что **не соответствует** действительности: virtio не передаёт тип носителя. SMART недоступен (`Unable to detect device type`)	`/sys/block/vda/...`, `smartctl`
Частота CPU	интерфейса `cpufreq` в госте нет: governor и Turbo нельзя ни прочитать, ни зафиксировать	`/sys/.../cpufreq`
PMU	аппаратные события (cycles, промахи кэша/TLB) в госте `<not supported>`	`perf stat`
Обратная запись	`dirty_ratio` 20, `dirty_background_ratio` 10, `dirty_expire_centisecs` 3000, `swappiness` 60	`sysctl`
Инструменты	Ubuntu clang 21.1.8, Python 3.14.4	`clang --version`

Таблица 1. Паспорт системы (полный вывод: `logs/passport.txt`)

2.2 Подготовка окружения (шаг 4.2)

Пункт	Решение и обоснование
Фоновая нагрузка	Графические приложения закрыты; load average ниже 1 (`uptime`), все vCPU простаивают ≈ 97 % времени (`mpstat`). Снимки состояния до и после каждой серии: `logs/snapshot_*.txt`.
Governor, Turbo	Не управляются из гостя (нет `cpufreq`); кроме того, macOS сама решает, на каком физическом ядре (P или E) выполняется vCPU. Зафиксировано как неконтролируемый фактор.
Закрепление	`taskset -c 2`. Все vCPU одинаково свободны; CPU0 дополнительно обслуживает прерывания сетевого virtio, поэтому он не выбран. Закреплённый vCPU по-прежнему планируется хостом.
Приоритет	`renice -n -5` для скрипта сбора; дочерние процессы его наследуют.
Состояние кэша	**Тёплый кэш** (по варианту). Свежие графы, `sync`, затем по одному неизмеряемому запуску каждой конфигурации; в течение серии кэш не сбрасывается.
Размер данных и ОЗУ	1 МиБ и 10 МиБ ≪ 3,3 ГиБ, поэтому оба файла целиком остаются в страничном кэше: 0 major page faults во всех 248 запусках финальной серии.
Доступ к perf	`kernel.perf_event_paranoid` = 1 установлен временно (по умолчанию 4), сбрасывается при перезагрузке.

Таблица 2. Чек-лист окружения

2.3 Тестовые данные и сборка

```
python3 ../util/graphgen.py -s 1M  --seed 427 --topology chain -b 0.5 --min-step-pages 2 --
verify -o graphs/graph-rand-1M.bin
python3 ../util/graphgen.py -s 10M --seed 427 --topology chain -b 0.5 --min-step-pages 2 --
verify -o graphs/graph-rand-10M.bin
clang -o out/graph_traverse src/graph_traverse.c     # флаги как в README
clang -o out/graph_traverse_mmap src/graph_traverse_mmap.c
```

Граф	Вершин	Страниц (4 КиБ)	Корень	sha256 (первые 12)
1M	43 689	256	9 668	`6513e809b696`
10M	436 905	2 560	252 929	`9cbbf5a3804a`

Таблица 3. Сгенерированные графы. Запись — 24 Б, заголовок — 40 Б, каждый переход ≥ 342 вершин (2 страницы); 50,2 % переходов вперёд / 49,8 % назад

Двукратная генерация дала одинаковые sha256, т.е. генерация воспроизводима. Так как `--write` изменяет файлы, графы были сгенерированы заново непосредственно перед финальной серией.

2.4 Ознакомительные измерения (шаг 4.3)

Одиночные запуски, один проход (полная таблица: `logs/baseline_step4.3.txt`):

Программа, флаги	1M, с	10M, с	vcs (10M)	Наблюдение
traverse	0,04	0,30	1 248	первые запуски после создания файла: эффект прогрева
traverse `--no-cache`	3,02	30,50	450 474	≈ 1 чтение с диска на вершину, ≈ 70 мкс каждое
traverse `--write`	0,03	0,37	1	запись лишь помечает страницы в ОЗУ как dirty, ожиданий нет
traverse `--write --no-cache`	9,10	90,51	1 326 711	3 дисковые операции на вершину (read-modify-write блока)
mmap, любые флаги	≤ 0,01	0,01–0,04	1–3	ниже разрешения `/usr/bin/time` (10 мс)

Таблица 4. Ознакомительные измерения: один запуск на конфигурацию

- **Системные вызовы** (`strace -c`, `graph_traverse --write 1`, 1M): 87 379 `lseek`, 43 691 `read`, 43 691 `write`, всего 174 793, т.е. 4 вызова на вершину (+1 чтение заголовка, +1 загрузка libc, +2 сообщения о прогрессе). Версия с mmap делает всего **37** вызовов и ни одного на вершину.

- **Page faults у mmap** (за вычетом ≈ 73 на старт программы): чтение — 16 для 1M и 162 для 10M, т.е. одна ошибка на 16 страниц (механизм ядра **fault-around**). Запись — 272 и 2 721 = одна ошибка записи на каждую страницу плюс ошибки чтения: первая запись в страницу должна пометить её как dirty. RSS у mmap растёт вместе с файлом (2,1 → 11,4 МБ), у `graph_traverse` остаётся 1,2 МБ.
- **Прогрев**: сразу после генерации чтение 10M заняло 0,30 с при 1 248 добровольных переключениях контекста; следующие запуски — 0,17 с при 1 переключении.
- **Шум**: при `stress-ng --cpu 4` время `graph_traverse --write 5` (10M) выросло с 1,69–1,70 с до 2,54–2,77 с (+57 %). Число вынужденных переключений выросло с 13–17 до 180–266, разброс — с 0,01 с до 0,23 с.
- **perf stat**: `task-clock` ≈ elapsed (процесс ограничен процессором); `cpu-migrations` 0–1 (единственная миграция — это `taskset`, переносящий себя на CPU 2 до `exec`); аппаратные события в госте не поддерживаются.

2.5 Гипотеза (шаг 4.4)

Оба файла помещаются в ОЗУ и целиком закэшированы, поэтому во время запуска диск не участвует. Размер файла может влиять только через кэши процессора и TLB: 256 и 2 560 страниц, посещаемых в случайном порядке.

- **H1 (read/write).** Стоимость вершины определяется 4 системными вызовами (≈ 190 нс каждый), которые не зависят от размера файла. Ожидание: 10M медленнее на вершину не более чем на **5 %**.
- **H2 (mmap).** Системных вызовов на вершину нет, стоимость — это доступ к памяти. Больший рабочий набор хуже помещается в кэши и TLB (в ВМ трансляция адресов двухуровневая). Ожидание: 10M медленнее на вершину на **10–25 %**, и разница проявится в **user** time.
- **H3.** mmap быстрее read/write на вершину более чем в 30 раз.

2.6 План эксперимента (шаг 4.5)

Пункт	Решение
Метрика запуска	Wall time одного процесса (`date +%s%N`, наносекундное разрешение) и производная **нс на вершину** = wall / (вершины × итерации). Также записываются user, sys, minflt, majflt, добровольные и вынужденные переключения (`/usr/bin/time`).
Равная работа	read/write: 1M × 50, 10M × 5 проходов; mmap: 1M × 2000, 10M × 200. Каждый запуск посещает ≈ одинаковое число вершин и длится 1,5–1,9 с; для mmap совпадает даже число page faults (544 тыс.).
Кэш	Тёплый, одинаковый для всех запусков (см. 2.2).
Порядок	N раундов. В каждом раунде 1M и 10M запускаются в случайном порядке (`shuf`), порядок записывается. Интерливинг защищает от дрейфа.
Прогрев	Первые **K = 2** раунда отбрасываются. Решение принято до финальной серии на основании эффекта прогрева из п. 2.4.
Выбросы	Правило 1,5 × IQR для каждой конфигурации; каждый удалённый запуск перечислен (приложение Б).
N	Пилот N = 10 → N ≈ (t·s/E)², E = 1 % от среднего (таблица 6). Выбрано **N = 60** (+ 2 раунда прогрева = 62), т.е. ≈ 2,5× от расчёта — запас на тяжёлые хвосты.
Измерение	В финальной серии только `/usr/bin/time` и `date`; `perf` и `strace` — только для характеризации.

Таблица 5. План эксперимента

Пилот, N = 10	среднее, с	s, с	CV	N для E = 1 %	Комментарий
read/write 1M	1,688	0,024	1,4 %	8	
read/write 10M	1,770	0,044	2,5 %	24	
mmap 1M	1,507	0,037	2,5 %	24	
mmap 10M	1,919	0,255	13,3 %	680	два ранних запуска (2,18 и 2,56 с); без них ≈ 24

Таблица 6. Пилотная серия и оценка N

Финальная серия (шаги 4.6–4.7) выполнена скриптом `collect.sh` (приложение B) сразу после свежего `gen_graphs.sh` и `sync`, со снимками `logs/snapshot_before_final.txt` и `logs/snapshot_after_final.txt`. Сырые данные: `data/final_read.csv` и `data/final_mmap.csv` (одна строка на запуск). Обработка: `analyze.py`.

3 Результаты

3.1 Этап 1: `graph_traverse` (read/lseek/write)

Размер	N	среднее, с	95 % ДИ, с	±ДИ	медиана, с	нс/вершину	user, с	sys, с
1M × 50	56	1,7910	1,7841–1,7979	0,38 %	1,7840	**819,9**	0,363	1,414
10M × 5	55	1,9289	1,9177–1,9401	0,58 %	1,9173	**883,0**	0,366	1,551

Таблица 7. Результаты этапа 1 (62 запуска на размер − 2 прогревочных − 4/5 выбросов)

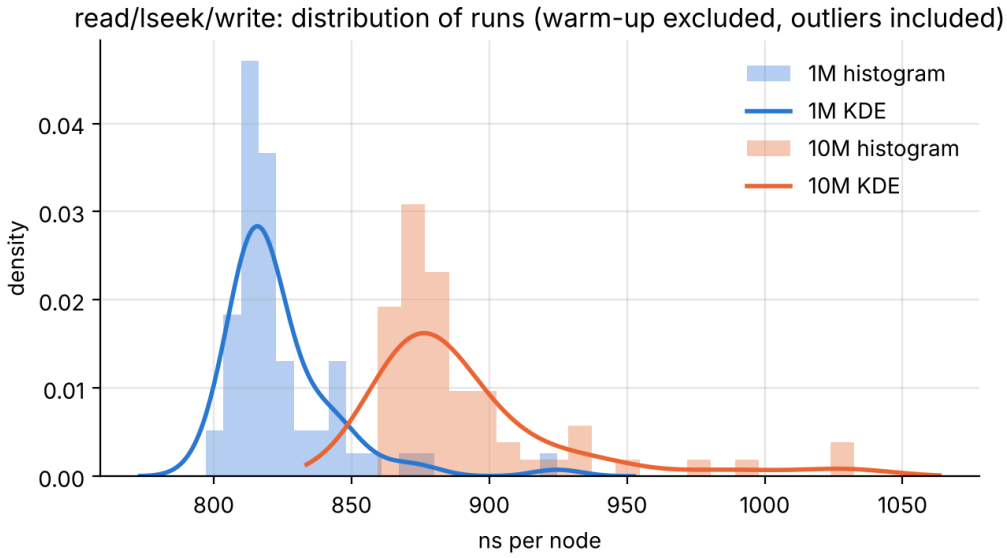


Рис. 1. Этап 1: распределение запусков, гистограмма + KDE (без прогрева, с выбросами)

Вывод по этапу 1 (шаг 4.9).

1. **Сравнение.** 10M медленнее на вершину на **7,7 %** (по медиане 7,5 %, с выбросами 8,3 %); t-тест Уэлча: t = 21,0, p ≈ 0. **Гипотеза H1 не подтвердилась**: эффект больше предсказанных 5 %. Механизм виден по разложению времени: **user time одинаковый** (0,363 и 0,366 с), **вся разница — в sys time** (+9,7 %). Замедляется не цикл программы, а ядро, потому что в этом способе доступа именно ядро работает с данными файла: копирует их из страницы кэша и в неё при каждом `read`/`write` и обновляет метаданные этой страницы. При 2 560 случайно посещаемых страницах эти данные хуже помещаются в кэши процессора, чем при 256. Без аппаратных счётчиков это объяснение подкреплено разложением user/sys, но не доказано прямым подсчётом промахов.
2. **Погрешность.** Полуширина ДИ — 0,38 % и 0,58 %. Интервалы не пересекаются: разность средних (0,138 с) ≈ в 12 раз больше большей полуширины (0,011 с).
3. **Достаточность.** По формуле для ±1 % нужно N = 9 (1M) и 19 (10M); у нас 56 и 55. Увеличение N сузит ДИ пропорционально 1/√N, но не изменит вывод.

3.2 Этап 2: `graph_traverse_mmap`

Размер	N	среднее, с	95 % ДИ, с	±ДИ	медиана, с	нс/вершину	user, с	sys, с
1M × 2000	57	1,4942	1,4821–1,5064	0,81 %	1,4723	**17,10**	1,177	0,304
10M × 200	53	1,7848	1,7663–1,8033	1,04 %	1,7546	**20,43**	1,448	0,323

Таблица 8. Результаты этапа 2 (62 запуска на размер − 2 прогревочных − 3/7 выбросов)

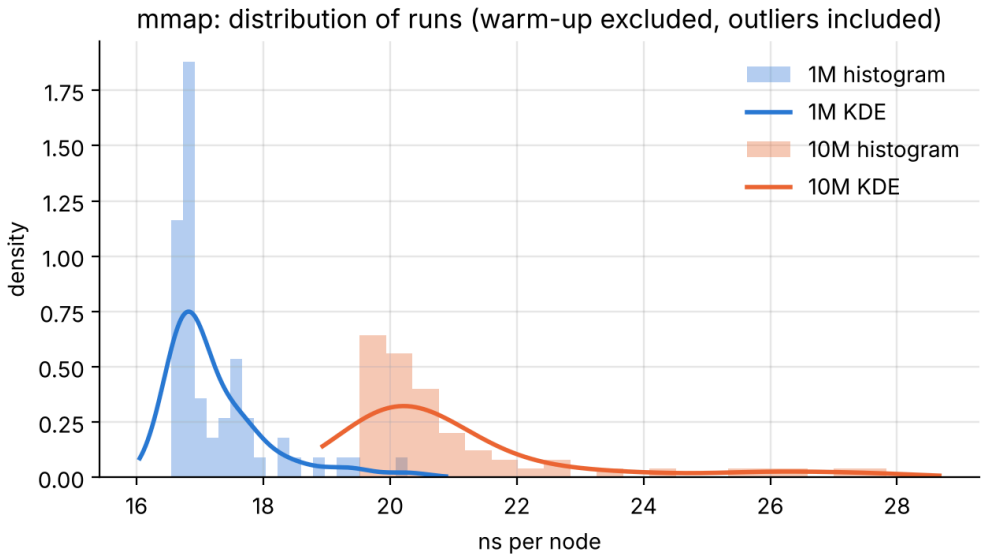


Рис. 2. Этап 2: распределение запусков, гистограмма + KDE

На запуск (финальная серия)	read/write 1M	read/write 10M	mmap 1M	mmap 10M
minor page faults	137	137	544 149	544 265
major page faults (все запуски)	0	0	0	0
добровольные переключения	1,02	1,04	1,35	1,17
вынужденные переключения	14,6	16,3	12,3	14,5
системных вызовов на проход	≈ 174 800	≈ 1 747 600	≈ 7	≈ 7

Таблица 9. Переключения контекста и page faults: read/write и mmap

Вывод по этапу 2.

1. **Сравнение.** 10M медленнее на вершину на **19,4 %** (по медиане 19,2 %, с выбросами 22,2 %; t = 26,4, p ≈ 0). **Гипотеза H2 подтвердилась.** Здесь разница в **user** time (+23 %), а sys time (+6 %) и число page faults практически равны по построению эксперимента. Программа сама обращается к отображённой странице, поэтому штраф кэшей/TLB от большего рабочего набора проявляется в пространстве пользователя.
2. **Page faults и переключения контекста.** У read/write ≈ 137 page faults на запуск (только старт программы) и ≈ 175 тыс. системных вызовов на проход. У mmap нет вызовов на вершину, зато за каждый проход — одна ошибка страницы на каждую затронутую страницу: 272 для 1M (256 ошибок записи + 16 ошибок чтения с fault-around) и 2 721 для 10M. Обработка 544 тыс. ошибок стоит ≈ 0,30 с sys time на запуск, ≈ 0,56 мкс на ошибку. Добровольных переключений ≈ 1 у обеих программ: никто не ждёт диск.
3. **Погрешность и достаточность.** ДИ (±0,81 % и ±1,04 %) не пересекаются. Для ±1 % нужно N = 38 и 57; у нас 57 и 53, т.е. для mmap 10M точность на уровне цели. Разность средних (0,291 с) ≈ в 16 раз больше большей полуширины, поэтому дополнительные измерения не изменят вывод.
4. **Гипотеза H3 подтвердилась**: mmap быстрее read/write на вершину в **48** (1M) и **43** (10M) раза.

Выводы отличаются от этапа 1, как и предсказывает задание: с системными вызовами эффект размера скрыт за большой фиксированной стоимостью и проявляется в ядре; при прямом доступе к памяти он в 2,5 раза больше в относительных величинах и проявляется в самой программе. В абсолютных величинах read/write теряет на вершину больше (+63 нс против +3,3 нс), так как каждую вершину ядро копирует дважды.

3.3 Этап 3: все серии вместе

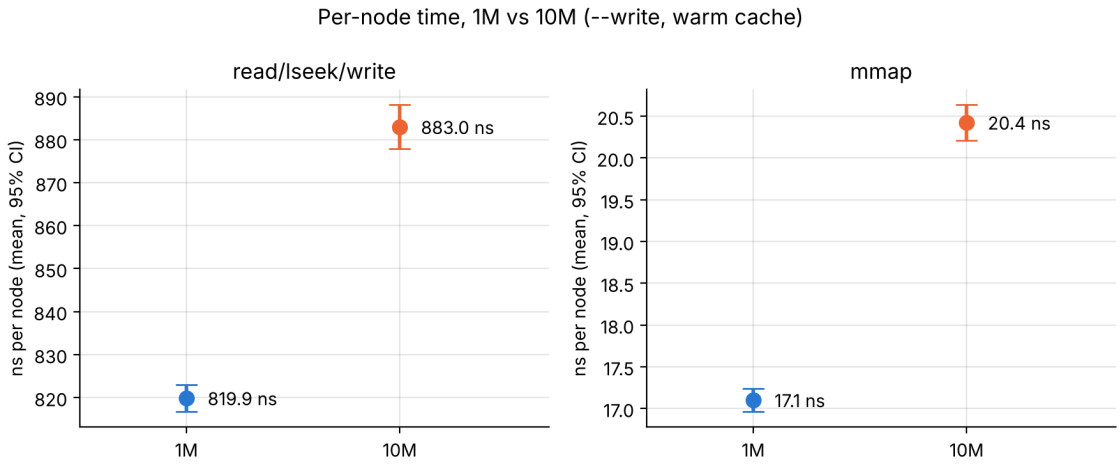


Рис. 3. Среднее время на вершину с 95 % доверительными интервалами

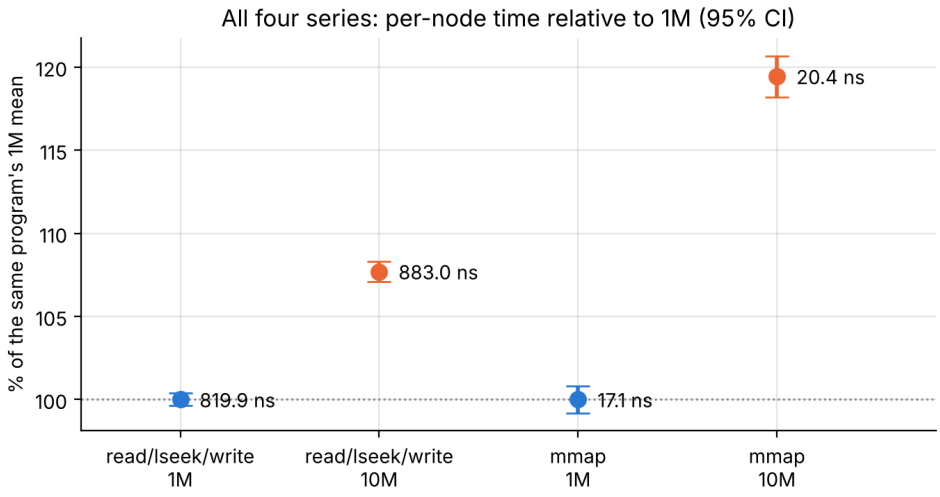


Рис. 4. Все четыре серии относительно среднего для 1M той же программы

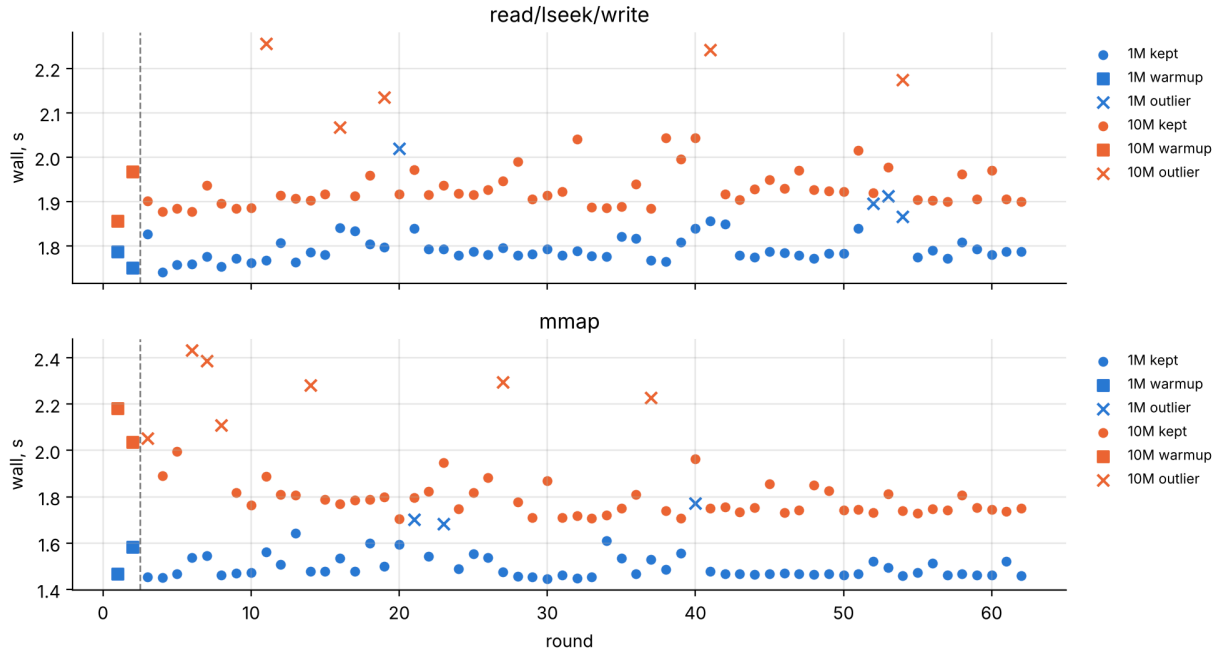


Рис. 5. Все запуски в порядке выполнения: прогрев (квадраты), выбросы (крестики)

Распределения не нормальные: тест Шапиро–Уилка даёт $p < 0{,}001$ для всех четырёх серий, с длинным **правым** хвостом редких медленных запусков. Поэтому рядом со средним приводятся медиана и IQR, а выводы проверены и с выбросами. У запусков-выбросов одновременно завышены и user, **и** sys, причём иногда они приходятся на соседние раунды обоих размеров сразу (read/write — раунды 52–54, mmap — раунды 35–40). Это замедление всей ВМ, а не одной конфигурации, и благодаря интерливингу оба размера поглотили его одинаково. Первый и второй запуски раунда отличаются меньше шума (все запуски без прогрева: read/write 855,5 и 860,8 нс, mmap 19,4 и 18,9 нс), так что порядок выполнения не смещает сравнение. mmap 10M прогревался дольше K = 2 раундов (раунды 1–8 повышены). K был зафиксирован заранее и не менялся; разность по медиане (19,2 %) показывает, что вывод от этого не зависит.

3.4 Интрузивность измерительных инструментов

Инструмент vs `/usr/bin/time`	**read/write**	**mmap**	**Вывод**
`perf stat` (10M, N = 20 пар)	1,9335 → 1,9418 с, +0,43 %, p = 0,70	1,8285 → 1,9230 с, +5,2 %, p = 0,18	при N = 20 не обнаруживается
`strace -c` (1M, 3 запуска)	0,19 → 79,1 с, ≈ 410×	0,40 → 0,87 с, ≈ 2,2×	огромная для кода с частыми системными вызовами

Таблица 10. Накладные расходы инструментов (`data/intrusive.csv`, `results/strace_overhead.txt`)

`strace` останавливает трассируемый процесс на входе и выходе каждого системного вызова (ptrace) и переключается на трассировщик: ≈ 874 тыс. вызовов × ≈ 90 мкс вместо ≈ 0,2 мкс. У mmap всего ≈ 7 вызовов на проход (≈ 3 500 в этом запуске, ≈ 130 мкс дополнительного времени каждый), поэтому абсолютные накладные расходы малы. Значит, интрузивность зависит не только от инструмента, но и от того, как измеряемая программа взаимодействует с ОС. Поэтому финальная серия измерялась только `/usr/bin/time` и `date`.

4 Заключение

1. **Ответ на вопрос варианта.** При использовании страничного кэша и файлах, много меньших ОЗУ, размер файла важен, хотя диск вообще не участвует: 10 МиБ медленнее 1 МиБ на вершину на 7,7 % при `read`/`write` и на 19,4 % при `mmap`. Эффект возникает в кэшах процессора и TLB и проявляется там, где выполняется код, работающий с данными: в ядре для read/write и в пространстве пользователя для mmap. На этой нагрузке mmap быстрее read/write в 43–48 раз.
2. **Критичные условия.** Страничный кэш: запись без него была в ≈ 245 раз медленнее (90,5 с против 0,37 с). Фоновая нагрузка: +57 % к wall time и разброс в 23 раза больше. Прогрев после создания файла, потребовавший `sync` и отбрасывания раундов. Равная работа на запуск: без неё сравнение показало бы лишь «в 10 раз больше вершин». Интерливинг: общие события шума затронули оба размера одинаково.
3. **Чем можно пренебречь.** Модель диска и виртуальный дисковый стек: 0 major page faults, т.е. диск не участвовал в измеряемых запусках. Накладные расходы `perf stat`, которые не обнаружены. Порядок запусков внутри раунда.
4. **Неконтролируемые факторы.** Частоту процессора и размещение vCPU на P/E-ядрах определяет macOS; аппаратные счётчики производительности в госте недоступны, поэтому объяснение через кэши/TLB опирается на разложение user/sys, а не на прямой подсчёт промахов.

5 Приложение А. Воспроизведение

```
cd lab/intro-exp
./build.sh                          # версия clang + обе программы
./gen_graphs.sh                     # графы + sha256 (должны совпасть с таблицей 3)
./passport.sh                       # logs/passport.txt
sync; ./snapshot.sh before_final
./collect.sh graph_traverse      62 data/final_read.csv
./collect.sh graph_traverse_mmap 62 data/final_mmap.csv
./snapshot.sh after_final
python3 analyze.py data/final_read.csv data/final_mmap.csv   # results/, figures/
./intrusive.sh 22 data/intrusive.csv
```

Файлы: `build.sh`, `gen_graphs.sh`, `passport.sh`, `snapshot.sh`, `collect.sh`, `intrusive.sh`, `analyze.py`; данные — в `data/` (пилот, финальная серия, интрузивность); логи — в `logs/` (ознакомительные измерения, perf/stress-ng, паспорт, снимки); таблицы — в `results/`; графики — в `figures/`.

6 Приложение Б. Удалённые выбросы (1,5 × IQR)

Конфигурация	Удалено (раунд: wall, с)	Кол-во	Общая черта
read/write 1M	20: 2,020; 52: 1,895; 53: 1,913; 54: 1,867	4	повышены и user, и sys
read/write 10M	11: 2,255; 16: 2,067; 19: 2,135; 41: 2,242; 54: 2,175	5	sys +6–19 %
mmap 1M	21: 1,701; 23: 1,684; 40: 1,772	3	повышены и user, и sys
mmap 10M	3: 2,054; 6: 2,432; 7: 2,388; 8: 2,108; 14: 2,282; 27: 2,296; 37: 2,228	7	в основном ранние раунды

Таблица 11. Выбросы (`results/outliers.csv`)

7 Приложение В. Скрипт сбора `collect.sh`

```
#!/usr/bin/env bash
# Usage: ./collect.sh <graph_traverse|graph_traverse_mmap> <N> <out.csv>
set -euo pipefail
cd "$(dirname "$0")"
PROG=$1; N=$2; OUT=$3
CORE=2
if [ "$PROG" = "graph_traverse" ]; then
  declare -A ITER=( [1M]=50   [10M]=5 )
else
  declare -A ITER=( [1M]=2000 [10M]=200 )
fi
declare -A NODES=( [1M]=43689 [10M]=436905 )
sudo renice -n -5 -p $$ > /dev/null
for s in 1M 10M; do                                  # прогрев кэша, без измерения
  taskset -c $CORE ./out/$PROG --write ${ITER[$s]} graphs/graph-rand-$s.bin 2>/dev/null
done
echo
"timestamp,program,size,flags,iter,nodes,core,round,pos,wall_s,user_s,sys_s,minflt,majflt,vcs,ics"
> "$OUT"
for round in $(seq 1 "$N"); do
  pos=0
  for s in $(printf "1M\n10M\n" | shuf); do          # случайный порядок в каждом раунде
    pos=$((pos + 1))
    t0=$(date +%s%N)
    /usr/bin/time -f "%U,%S,%R,%F,%w,%c" -o /tmp/collect_time.txt \
      taskset -c $CORE ./out/$PROG --write ${ITER[$s]} graphs/graph-rand-$s.bin 2>/dev/null
    t1=$(date +%s%N)
    wall=$(awk -v a="$t0" -v b="$t1" 'BEGIN { printf "%.6f", (b - a) / 1e9 }')
    echo "$(date +%s),$PROG,$s,--write,${ITER[$s]},${NODES[$s]},$CORE,$round,$pos,$wall,
$(cat /tmp/collect_time.txt)" >> "$OUT"
  done
  echo "round $round/$N done"
done
```